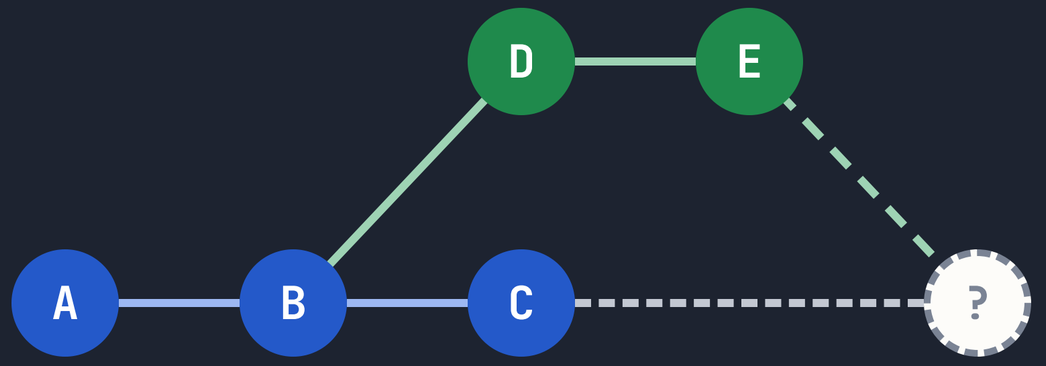
KANT AX/AI 엔지니어 트랙 · 08

합치는 방법, 눈으로 비교하기

fast-forward, merge, rebase, squash



갈라진 두 길을 어떻게 이을까요

오늘의 순서

1	갈라지지 않았을 때	fast-forward, --no-ff	7–8쪽
2	갈라졌을 때	merge, rebase, squash	9–16쪽
3	한눈에 비교	네 가지 결과 그림과 표	17–18쪽
4	git pull은 무엇을 하나	pull 설정 세 가지	19–24쪽
5	GitHub PR의 Merge 버튼	버튼 세 개와 명령	25–26쪽
6	충돌과 오류	충돌, push한 브랜치, 막힐 때	27–30쪽
7	실제 상황 예제	PR 하나를 올리고 합치기까지	31–33쪽

3–6쪽은 왜 배우는지와 오늘의 예시, 34쪽은 한 장 정리예요.

들어가기

같은 브랜치를 합쳤는데 기록이 달라요

merge로 합친 폴더

```
$ git log --oneline
8818bb8 (HEAD -> main) Merge branch 'feature/login-button'
d51d818 docs: README 추가
3573c9a (feature/login-button) fix: 버튼 글자 고치기
6a9792d feat: 로그인 버튼 추가
6d19f01 feat: 아이디 입력 추가
9f452aa feat: 로그인 화면 추가
$ cat login.md
# 로그인
아이디
[로그인] 버튼
```

squash로 합친 폴더

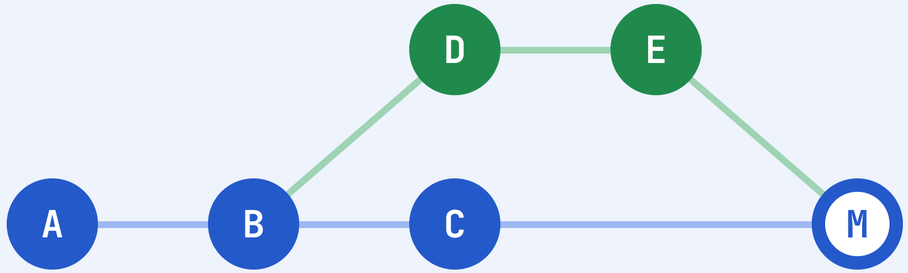
```
$ git log --oneline
8a6fe21 (HEAD -> main) feat: 로그인 버튼 추가
d51d818 docs: README 추가
6d19f01 feat: 아이디 입력 추가
9f452aa feat: 로그인 화면 추가
$ cat login.md
# 로그인
아이디
[로그인] 버튼
```

기록은 6줄과 4줄인데, 파일 내용은 똑같아요.

들어가기

합치기는 세 곳에서 일어나요

내 컴퓨터



1 **git merge, git rebase**
git merge --squash
1~3부

`git pull`

2 **받아 와서**
합치기
4부

GitHub

feat: 로그인 버튼 추가 #1

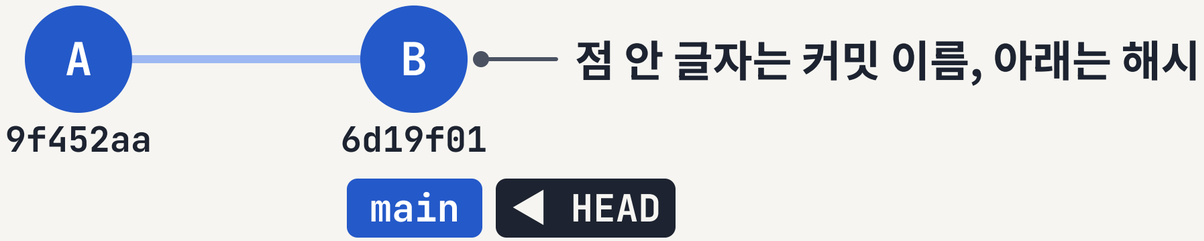
kim wants to merge 2 commits into main

Merge pull request ▼

3 **PR의 Merge 버튼**
5부

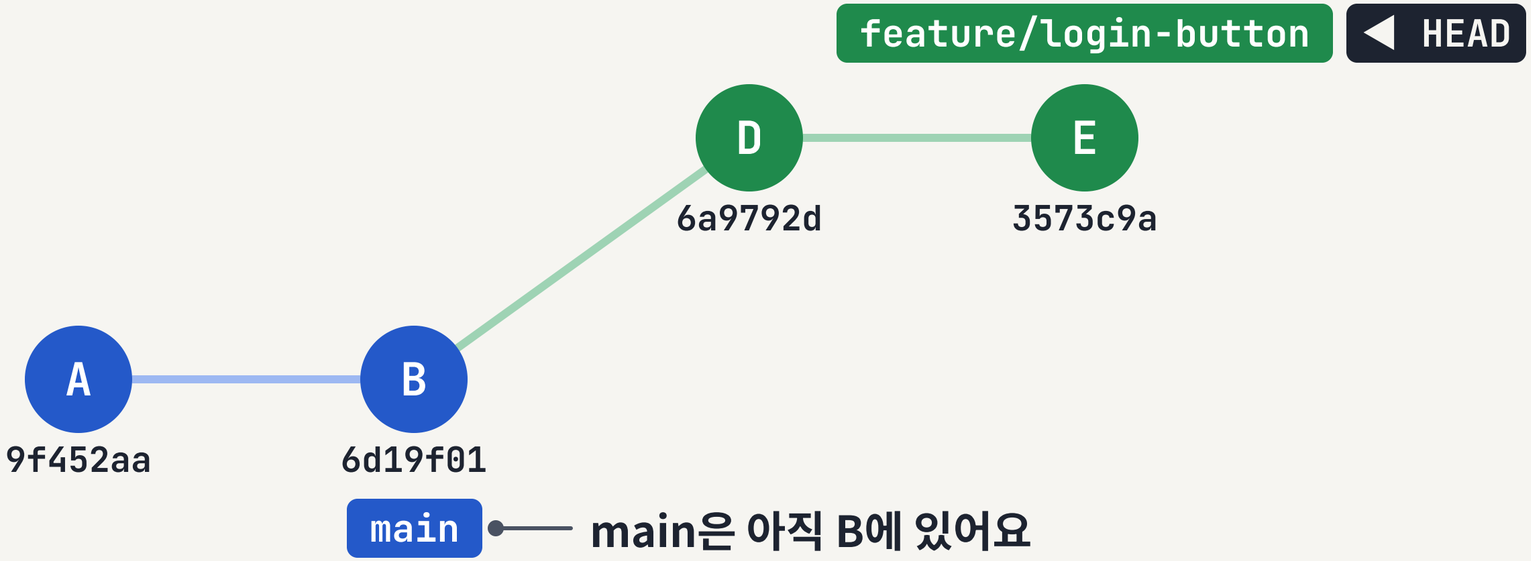
A, B까지 main에 있어요

```
$ git log --oneline
6d19f01 (HEAD -> main) feat: 아이디 입력 추가
9f452aa feat: 로그인 화면 추가
```



브랜치에서 D, E를 커밋해요

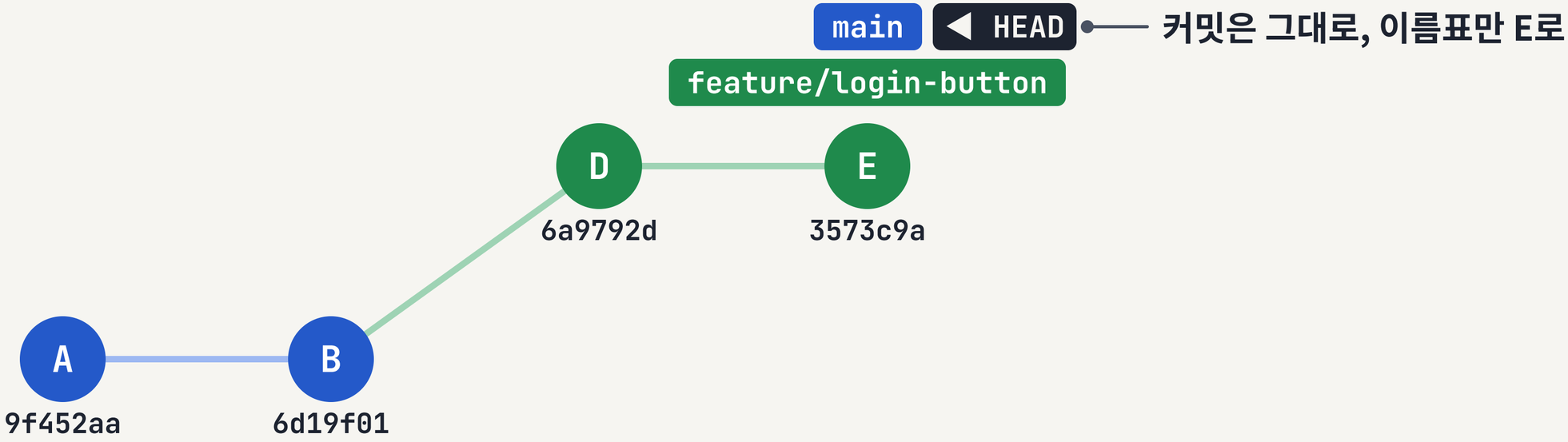
```
$ git checkout -b feature/login-button
$ git commit -m "feat: 로그인 버튼 추가"
[feature/login-button 6a9792d] feat: 로그인 버튼 추가
$ git commit -m "fix: 버튼 글자 고치기"
[feature/login-button 3573c9a] fix: 버튼 글자 고치기
```



1. 갈라지지 않았을 때

fast-forward: main 이름표만 앞으로 가요

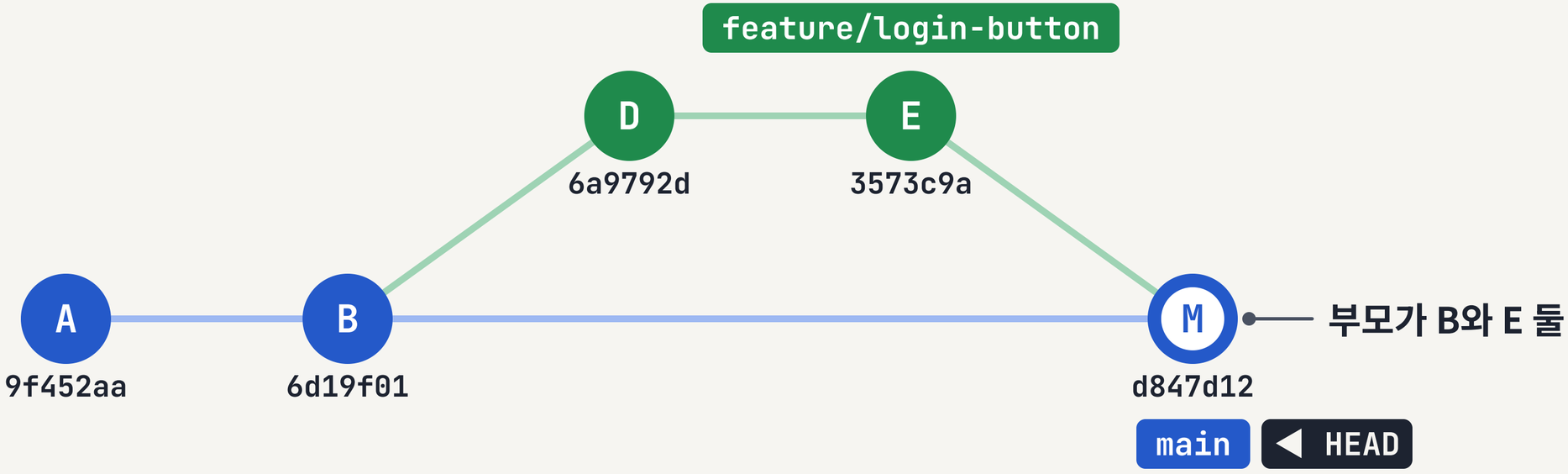
```
$ git checkout main
$ git merge feature/login-button
Updating 6d19f01..3573c9a
Fast-forward
```



1. 갈라지지 않았을 때

--no-ff: 그래도 합치기 커밋을 만들어요

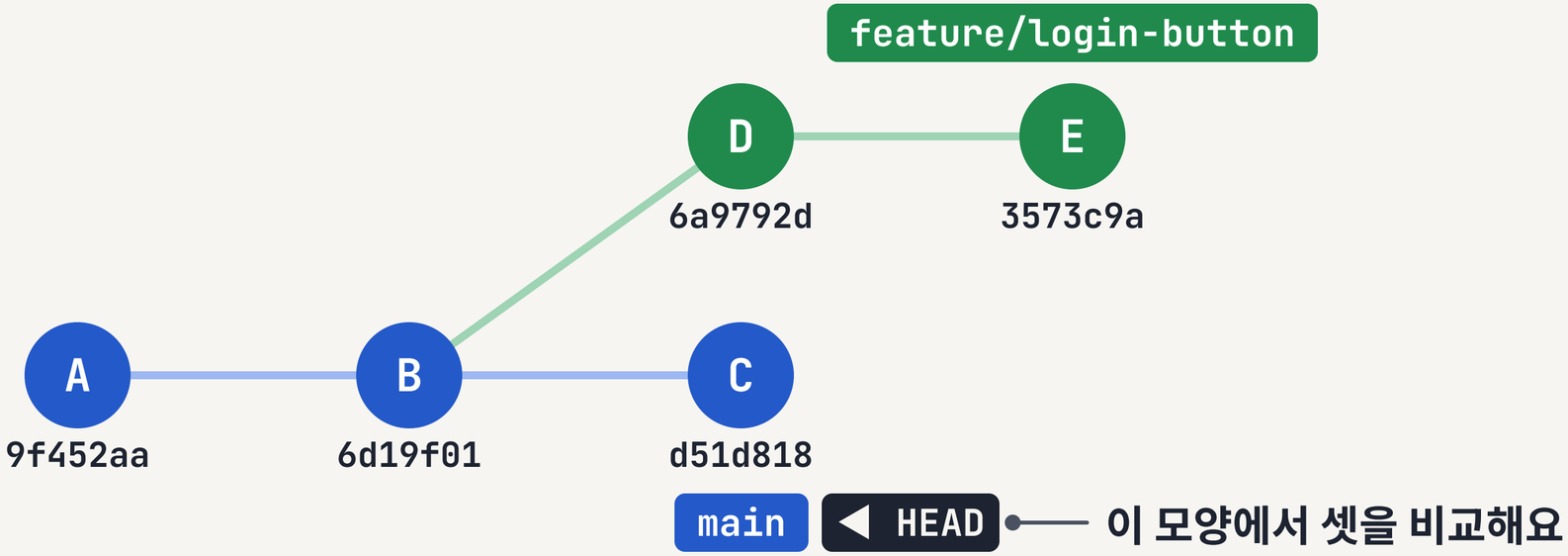
```
$ git merge --no-ff feature/login-button
Merge made by the 'ort' strategy.
$ git log --oneline
d847d12 (HEAD -> main) Merge branch 'feature/login-button'
```



2. 갈라졌을 때

main에 C가 생기면 길이 갈라져요

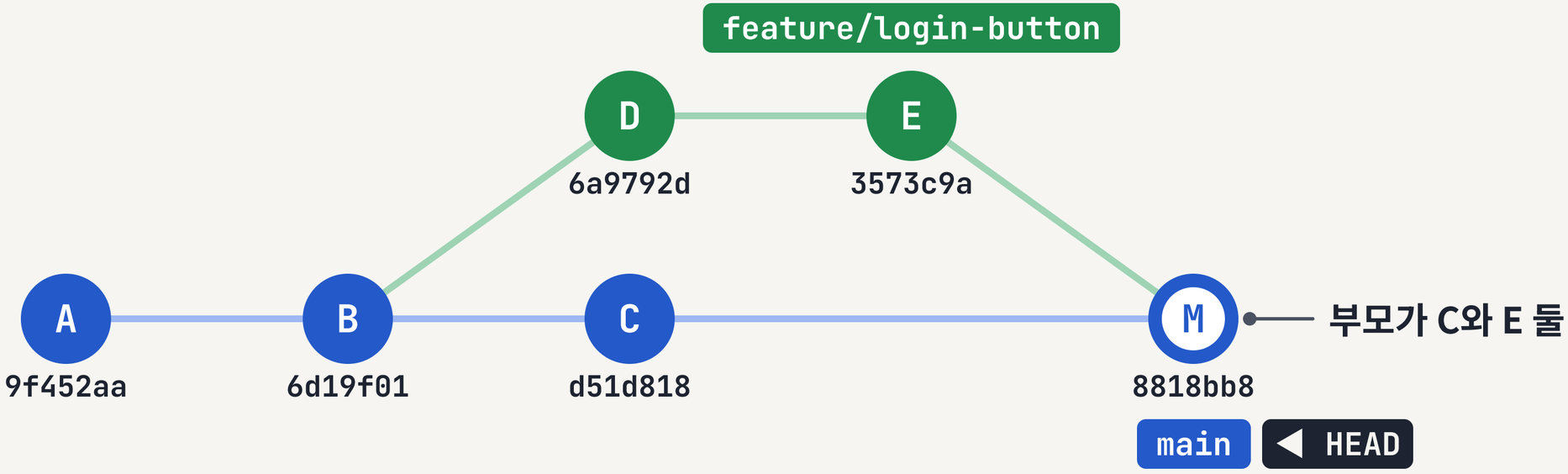
```
$ git add README.md
$ git commit -m "docs: README 추가"
[main d51d818] docs: README 추가
```



2. 갈라졌을 때

merge: 두 길을 잇는 합치기 커밋이 생겨요

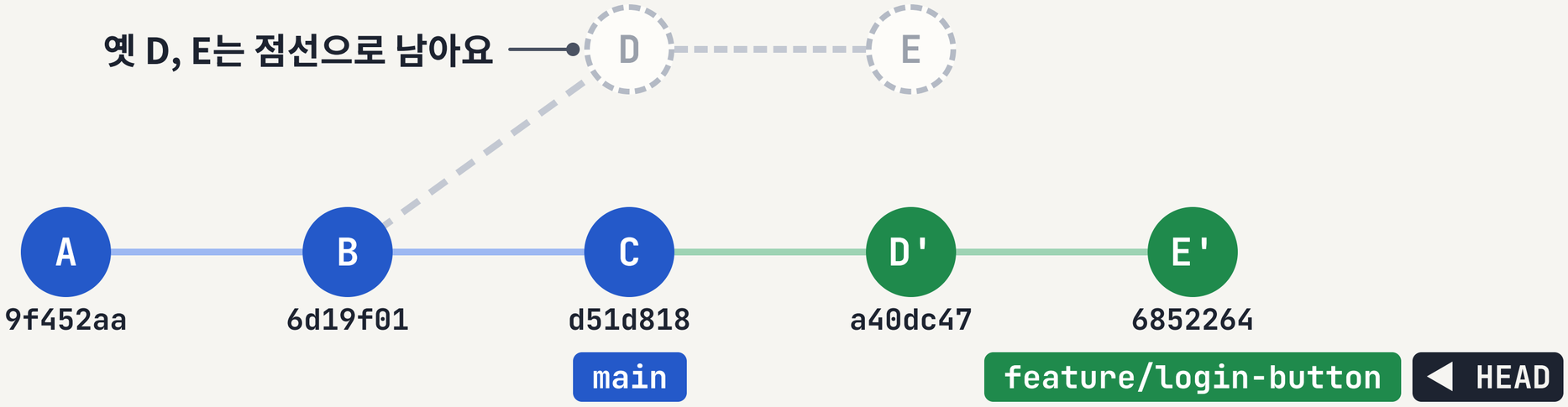
```
$ git merge feature/login-button
Merge made by the 'ort' strategy.
$ git log --oneline
8818bb8 (HEAD -> main) Merge branch 'feature/login-button'
```



2. 갈라졌을 때

rebase: D, E를 C 뒤로 옮겨 붙여요

```
$ git checkout feature/login-button
$ git rebase main
Successfully rebased and updated refs/heads/feature/login-button.
```



2. 갈라졌을 때

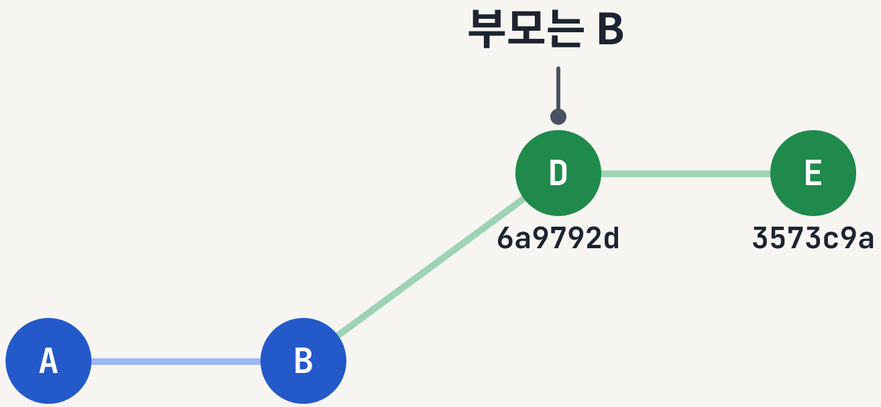
rebase하면 커밋 해시가 바뀌어요

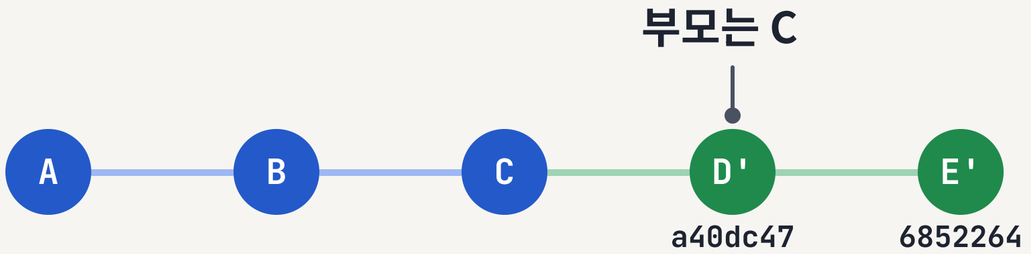
rebase 전

```
$ git log --oneline
3573c9a (HEAD -> feature/login-button) fix: 버튼 글자 고치기
6a9792d feat: 로그인 버튼 추가
6d19f01 feat: 아이디 입력 추가
9f452aa feat: 로그인 화면 추가
```

rebase 뒤

```
$ git log --oneline
6852264 (HEAD -> feature/login-button) fix: 버튼 글자 고치기
a40dc47 feat: 로그인 버튼 추가
d51d818 (main) docs: README 추가
6d19f01 feat: 아이디 입력 추가
```



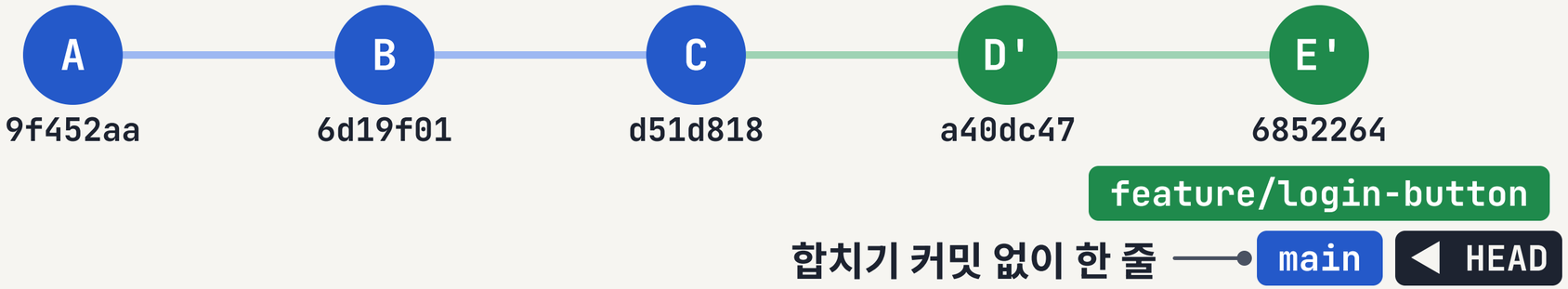


부모가 B에서 C로 바뀌어서 해시가 새로 나와요. 메시지와 내용은 그대로예요.

2. 갈라졌을 때

main에서 merge하면 fast-forward로 끝나요

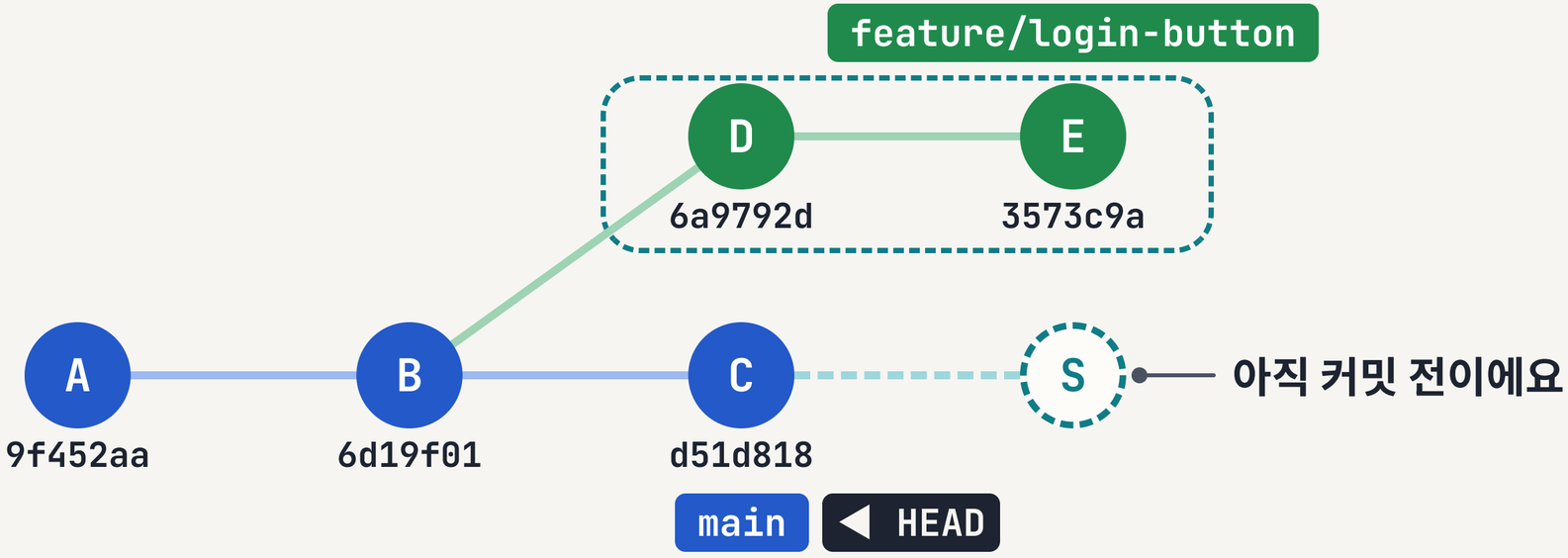
```
$ git checkout main
$ git merge feature/login-button
Updating d51d818..6852264
Fast-forward
```



2. 갈라졌을 때

squash: D, E의 변경을 모아 대기실에 올려요

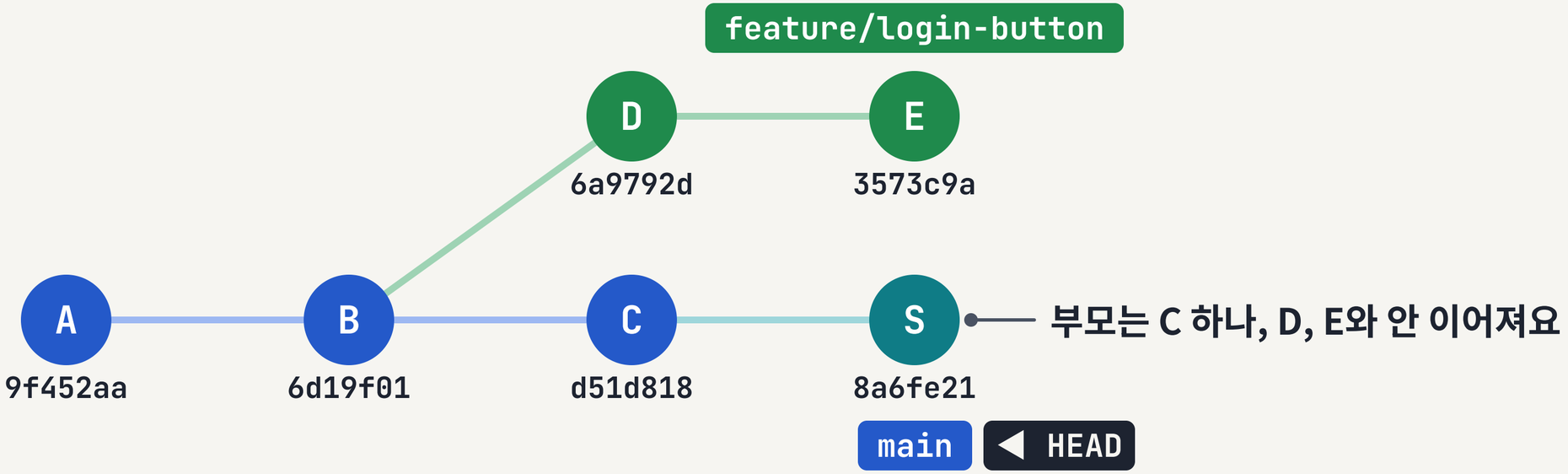
```
$ git merge --squash feature/login-button
Automatic merge went well; stopped before committing as requested
Squash commit -- not updating HEAD
```



2. 갈라졌을 때

커밋하면 S 하나가 생겨요

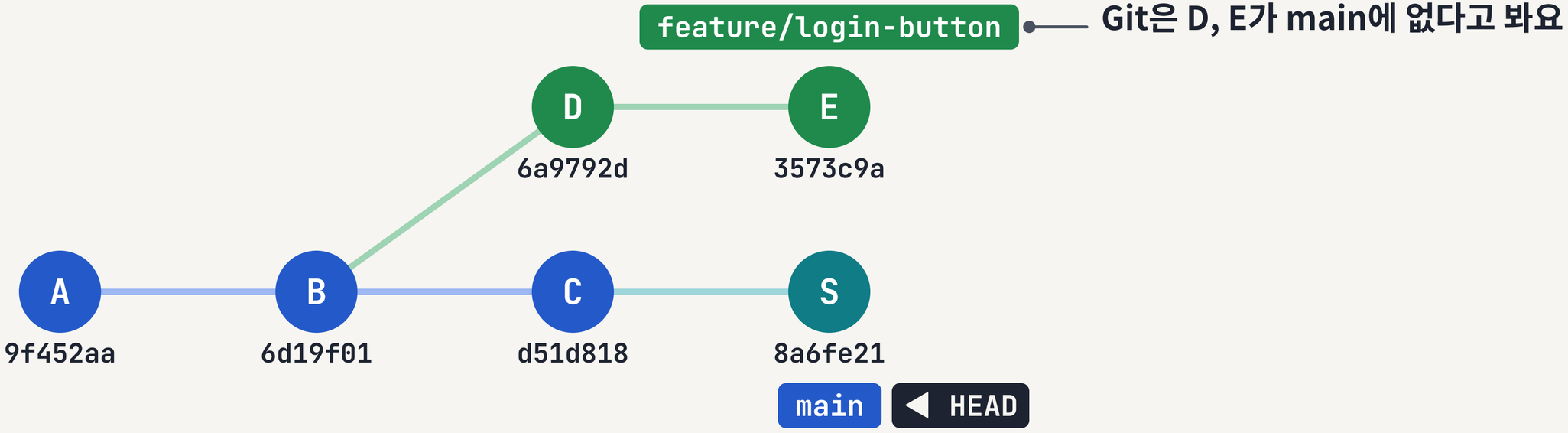
```
$ git commit -m "feat: 로그인 버튼 추가"
[main 8a6fe21] feat: 로그인 버튼 추가
```



2. 갈라졌을 때

squash 뒤에는 브랜치가 -d로 안 지워져요

```
$ git branch -d feature/login-button
error: the branch 'feature/login-button' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feature/login-button'
```



3. 한눈에 비교

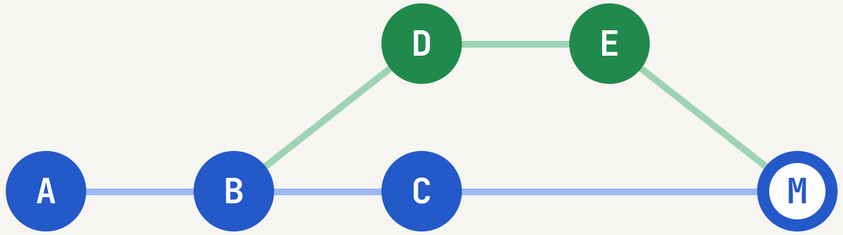
합치기 커밋과 해시에서 갈려요

fast-forward 갈라지지 않았을 때



새 커밋 없이 이름표만 앞으로

merge --no-ff도 같아요



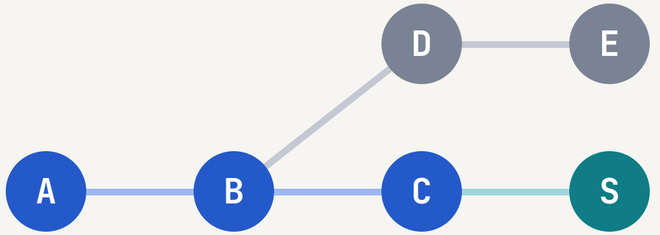
합치기 커밋 M, D와 E는 그대로

rebase 옮겨 붙인 뒤 fast-forward



한 줄, D'와 E'는 새 해시

squash 변경을 모아 커밋 하나로



한 줄, main에는 S 하나만

3. 한눈에 비교

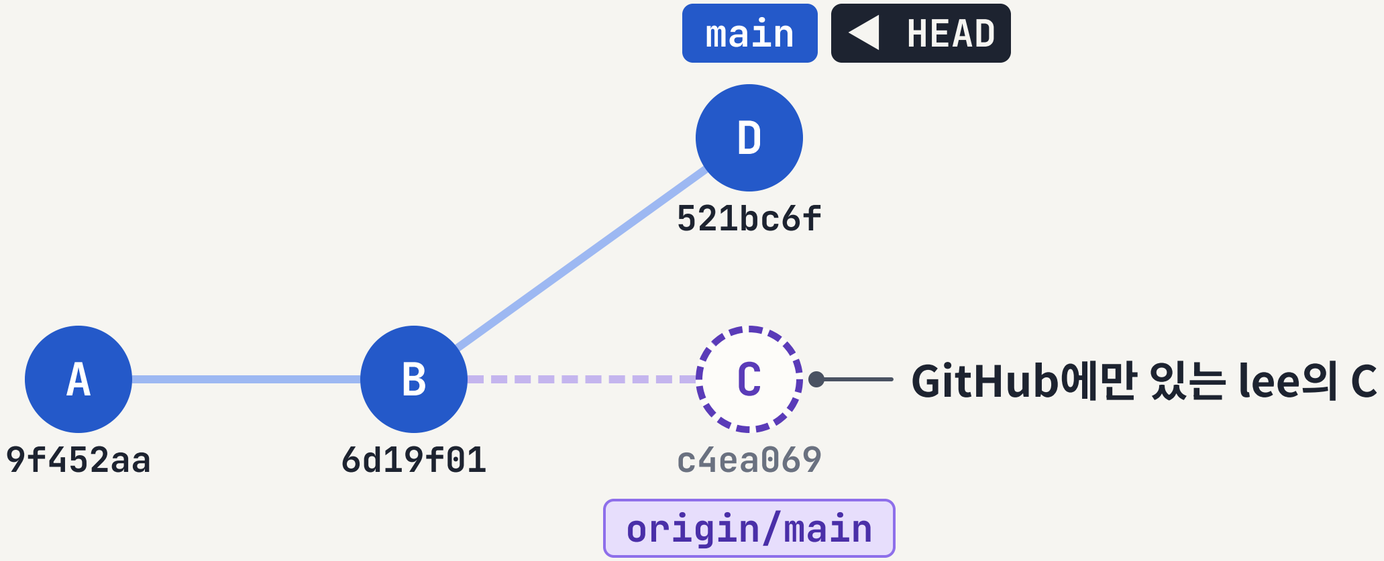
표로 보면 해시 칸이 달라요

방법	명령	새로 생기는 커밋	D, E 해시	main 기록
fast-forward	`git merge 브랜치`	없음	그대로	한 줄
merge	`git merge 브랜치` `git merge --no-ff 브랜치`	합치기 커밋 M	그대로	두 줄이 만나요
rebase	`git rebase main` `(브랜치에서)`	D', E'	**바뀌어요**	한 줄
squash	`git merge --squash 브랜치` `git commit -m "..."`	S 하나	**main에 없어요**	한 줄, 커밋 하나

4. git pull은 무엇을 하나

pull도 합치기예요

```
$ git fetch origin
$ git log --oneline
521bc6f (HEAD -> main) feat: 로그인 버튼 추가
$ git log --oneline origin/main
c4ea069 (origin/main) docs: README 추가
```



4. git pull은 무엇을 하나

설정이 없으면 pull이 멈추고 셋 중 고르라고 해요

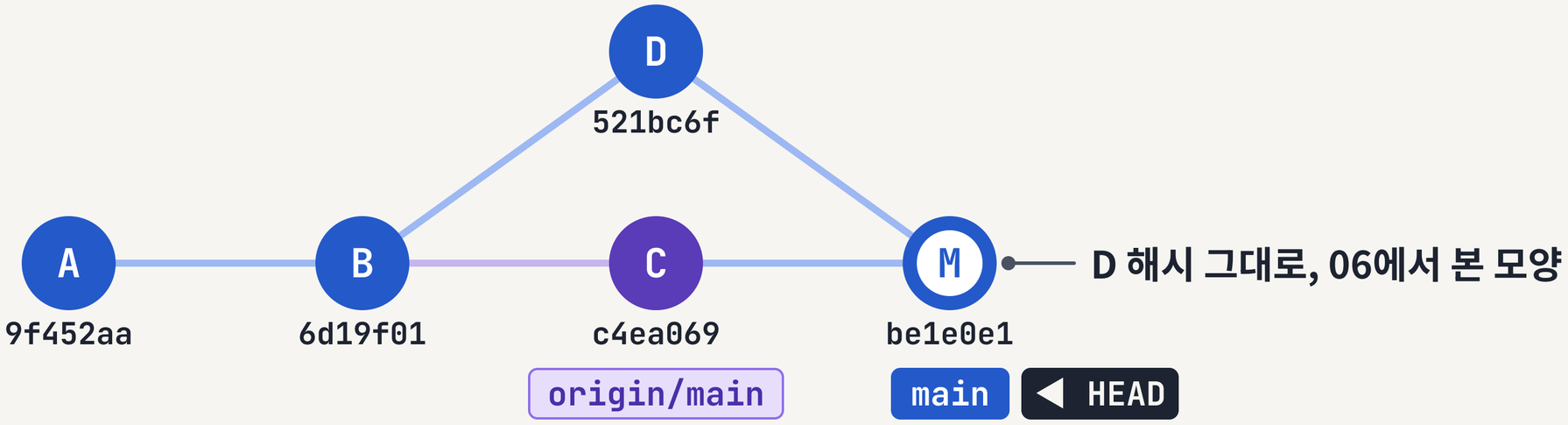
```
$ git pull origin main
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
fatal: Need to specify how to reconcile divergent branches.
```

merge로 합쳐요 · 수업 설정
rebase로 옮겨 붙여요
fast-forward만 받아요

06에서 git config --global pull.rebase false로 첫째 줄을 골랐어요.

pull.rebase false: 합치기 커밋으로 합쳐요

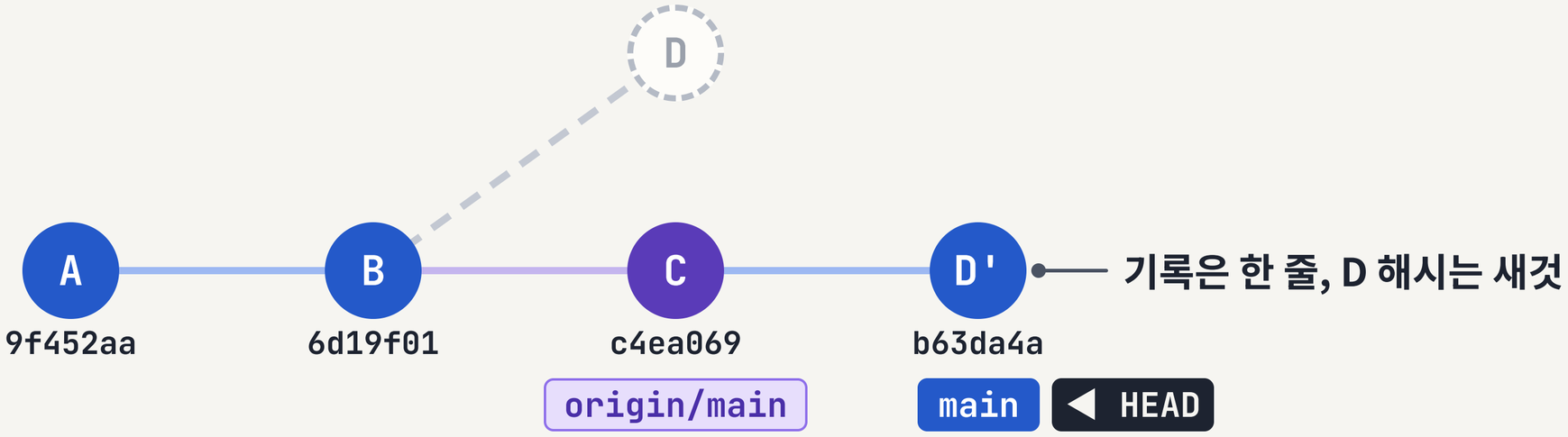
```
$ git config --global pull.rebase
false
$ git pull origin main
Merge made by the 'ort' strategy.
```



4. git pull은 무엇을 하나

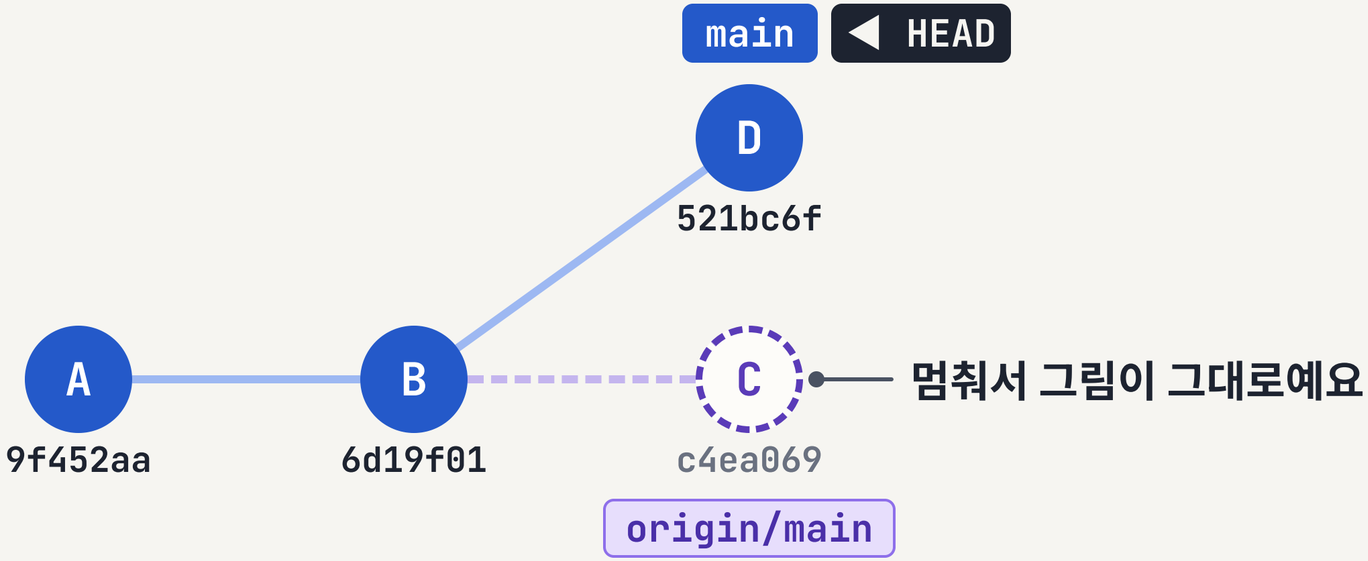
pull.rebase true: 내 커밋을 C 뒤로 옮겨요

```
# pull.rebase true인 컴퓨터
$ git pull origin main
Successfully rebased and updated refs/heads/main.
$ git log --oneline
b63da4a (HEAD -> main) feat: 로그인 버튼 추가
```



pull.ff only: 갈라졌으면 받지 않아요

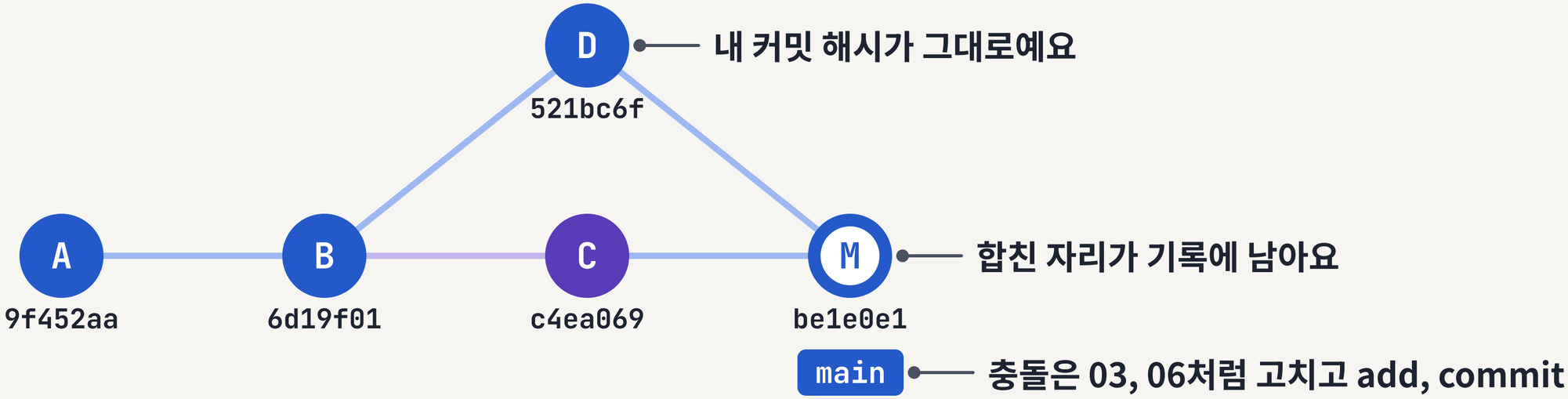
```
# pull.ff only인 컴퓨터
$ git pull origin main
fatal: Not possible to fast-forward, aborting.
```



4. git pull은 무엇을 하나

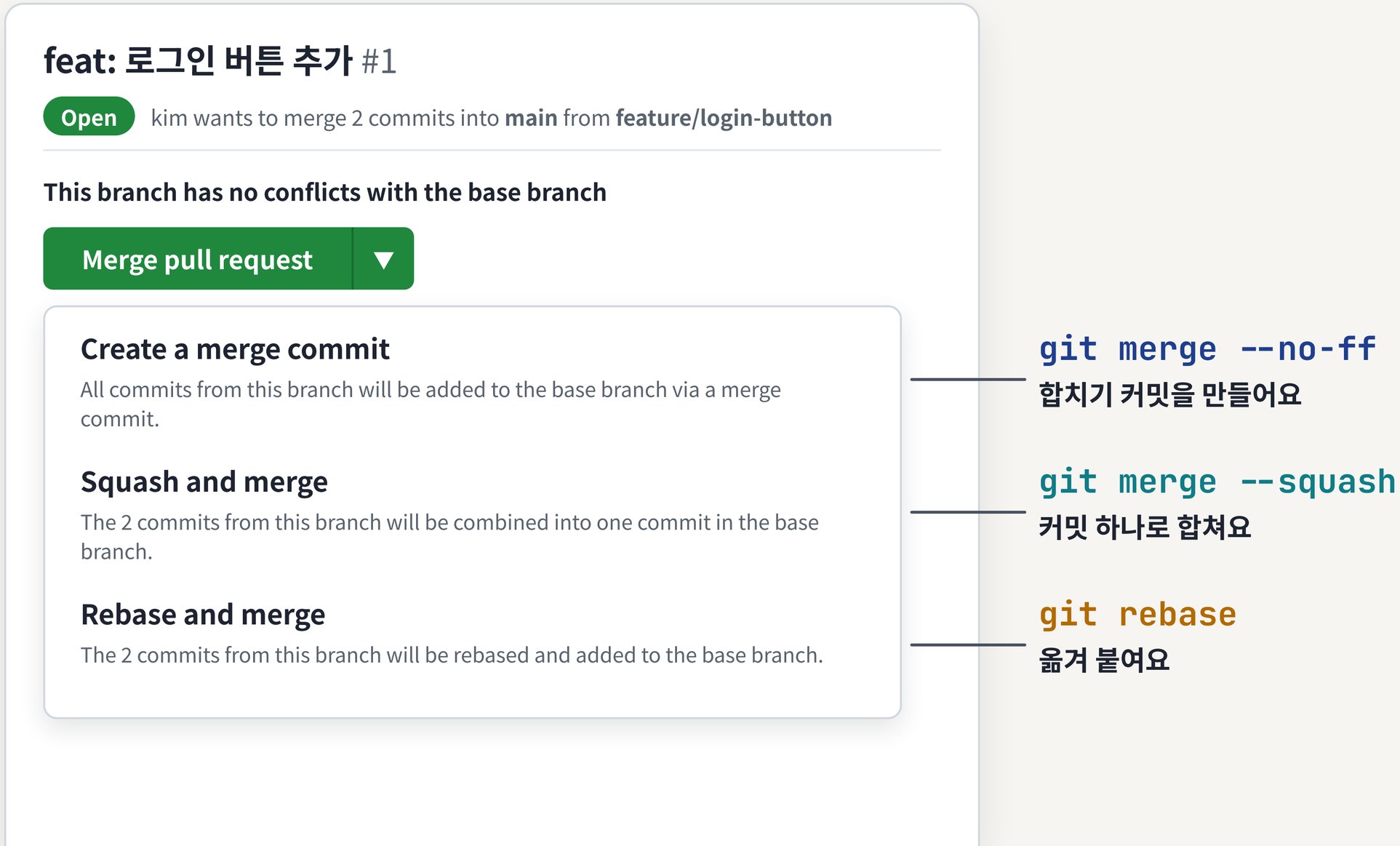
수업은 pull.rebase false로 통일해요

```
$ git config --global pull.rebase false
```



5. GitHub PR의 Merge 버튼

PR의 Merge 버튼은 세 가지예요



`git merge --no-ff`
합치기 커밋을 만들어요

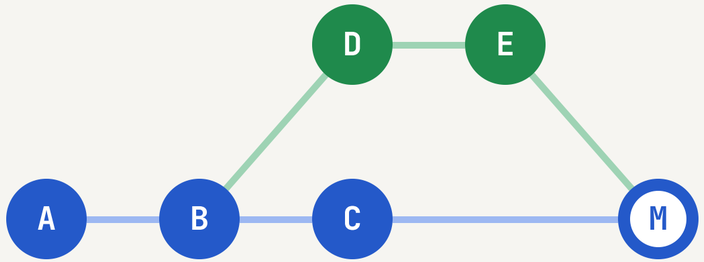
`git merge --squash`
커밋 하나로 합쳐요

`git rebase`
옮겨 붙여요

5. GitHub PR의 Merge 버튼

버튼마다 GitHub의 main 기록이 달라요

Create a merge commit




```
Merge pull request #1 from
kim/feature/login-button
```

Squash and merge




```
feat: 로그인 버튼 추가 (#1)
```

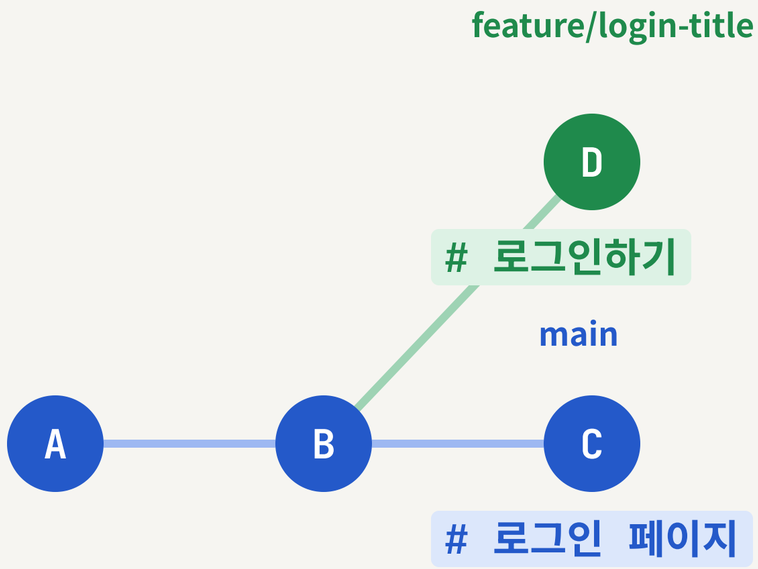
Rebase and merge




```
D', E' 해시가 새로 생겨요
```

6. 충돌과 오류

충돌은 어느 방법이든 같은 줄이면 나요




```
# main에서
$ git merge feature/login-title
CONFLICT (content): Merge conflict in login.md

# feature/login-title에서
$ git rebase main
CONFLICT (content): Merge conflict in login.md

# main에서
$ git merge --squash feature/login-title
CONFLICT (content): Merge conflict in login.md
```

같은 줄을 다르게 고쳤으면 Git은 방법과 상관없이 물어봐요.

6. 충돌과 오류

rebase 중 충돌은 add 뒤 --continue로 이어 가요

```
$ cat login.md
<<<<<<< HEAD
# 로그인 페이지
=======
# 로그인하기
>>>>>>> bf59624 (feat: 로그인 제목 바꾸기)
아이디
```

HEAD 쪽이 main 내용

아래쪽이 옮기던 내 커밋

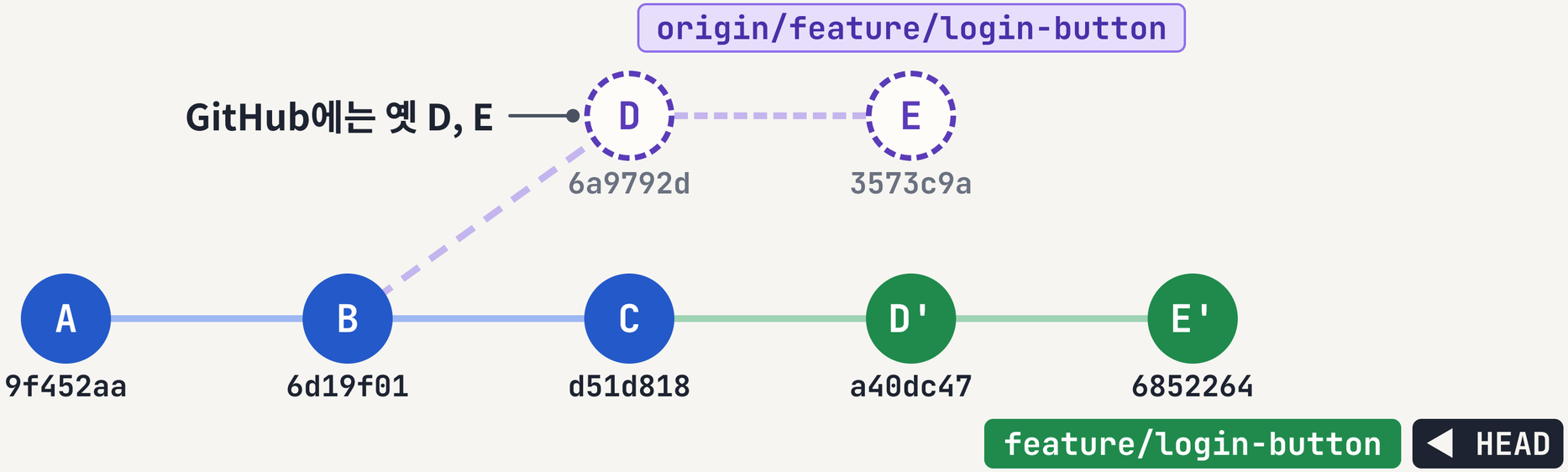
1. 표시 세 줄을 지우고 남길 내용으로 고쳐요
2. `git add login.md`
3. `git rebase --continue`

헷갈리면 그만두기

```
git rebase --abort
```

push한 브랜치는 rebase하지 않아요

```
$ git rebase main
$ git push origin feature/login-button
 ! [rejected]        feature/login-button -> feature/login-button (non-fast-forward)
```



6. 충돌과 오류

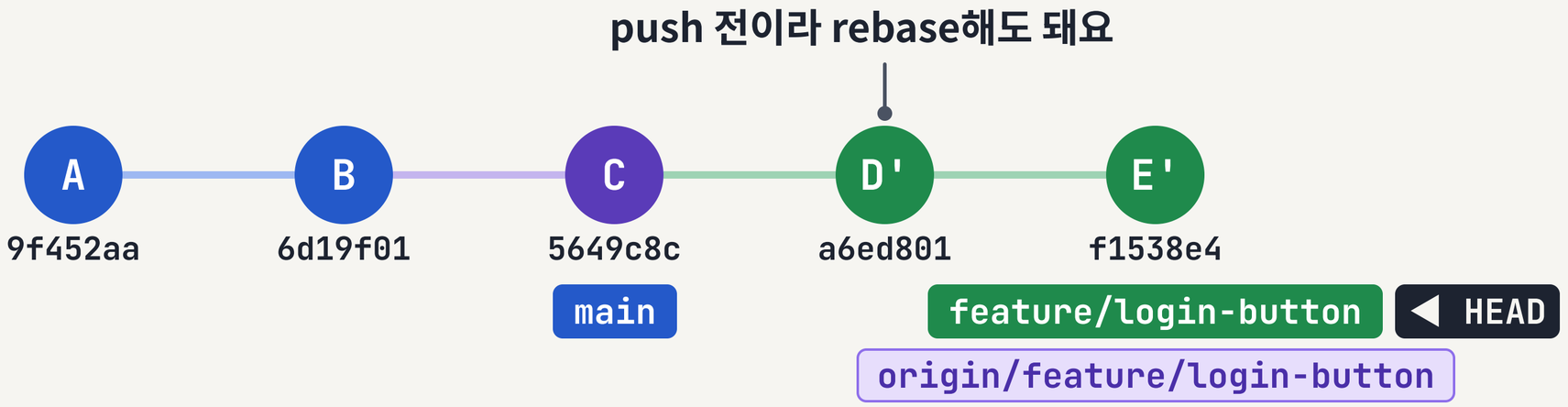
이 메시지가 나오면 이렇게 해요

`fatal: Need to specify how to reconcile divergent branches.` pull 설정이 없어요	`git config --global pull.rebase false`
`fatal: Not possible to fast-forward, aborting.` pull.ff only가 설정돼 있어요	`git config --global --unset pull.ff`
`error: the branch '...' is not fully merged.` squash나 rebase로 합쳤어요. 합친 걸 확인하고	`git branch -D 브랜치`
`error: could not apply ... (rebase 중 CONFLICT)` 고치고 add 뒤 이어 가요. 그만두려면 --abort	`git rebase --continue`
`! [rejected] ... (non-fast-forward) rebase 뒤` push한 브랜치를 rebase했어요. pull하지 말고	**강사에게 물어봐요**

7. 실제 상황 예제

PR을 올리기 전에 main이 앞서 갔어요

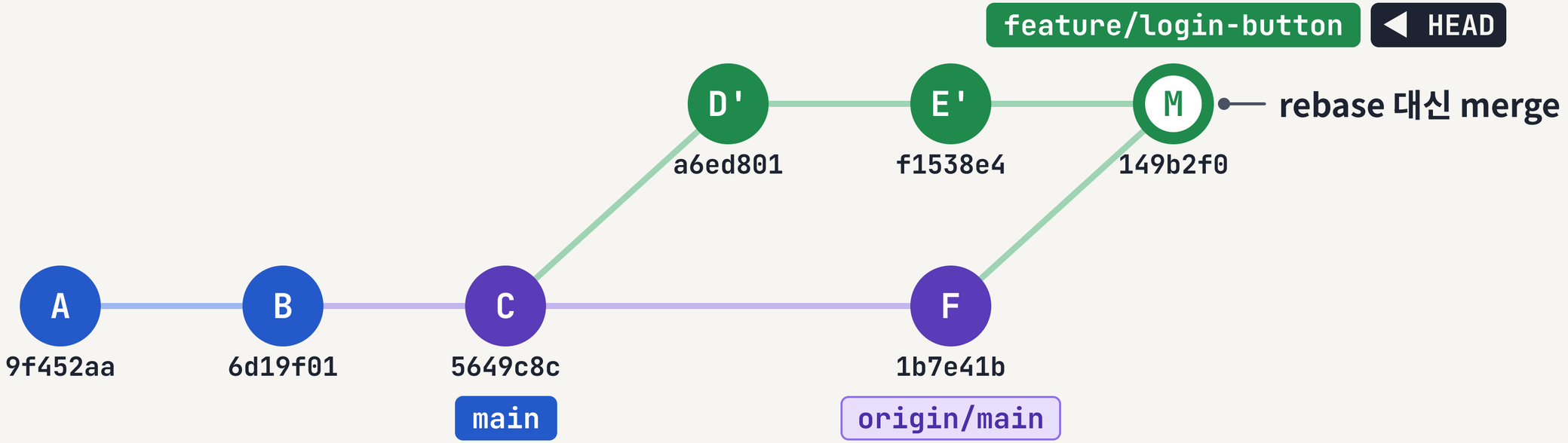
```
$ git checkout main
$ git pull origin main
Updating 6d19f01..5649c8c
Fast-forward
$ git checkout feature/login-button
$ git rebase main
Successfully rebased and updated refs/heads/feature/login-button.
$ git push origin feature/login-button
 * [new branch]      feature/login-button -> feature/login-button
```



7. 실제 상황 예제

PR을 올린 뒤에 main이 또 앞서 갔어요

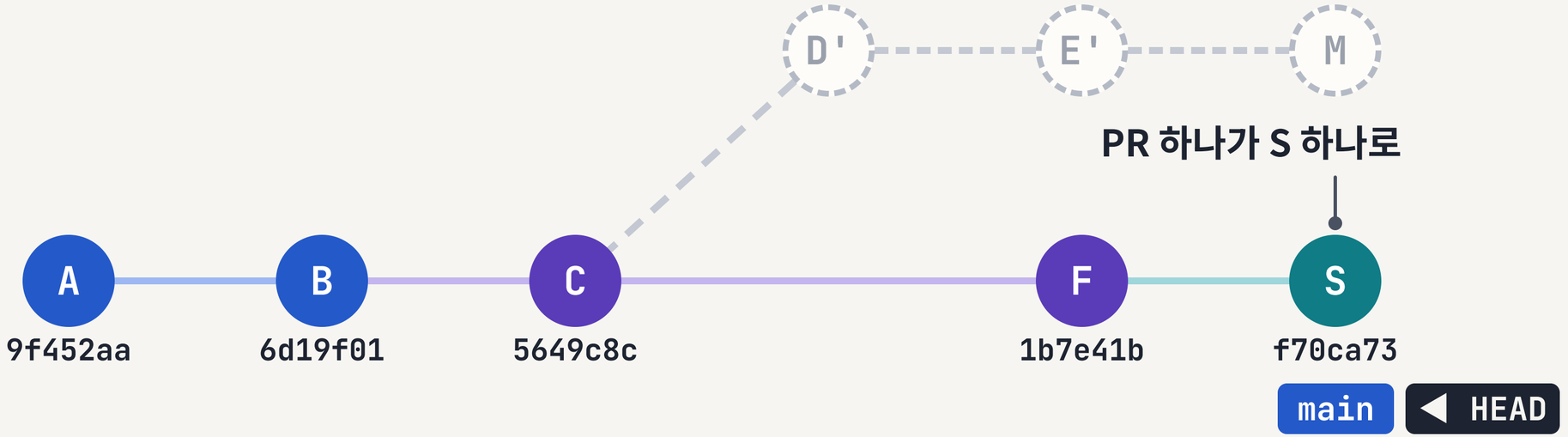
```
# feature/login-button에서
$ git pull origin main
Merge made by the 'ort' strategy.
$ git push origin feature/login-button
   f1538e4..149b2f0  feature/login-button -> feature/login-button
```



7. 실제 상황 예제

팀이 Squash and merge로 PR을 합쳤어요

```
$ git checkout main
$ git pull origin main
Updating 5649c8c..f70ca73
Fast-forward
$ git branch -d feature/login-button
error: the branch 'feature/login-button' is not fully merged.
$ git branch -D feature/login-button
Deleted branch feature/login-button (was 149b2f0).
```



정리

언제 무엇으로 합치나요

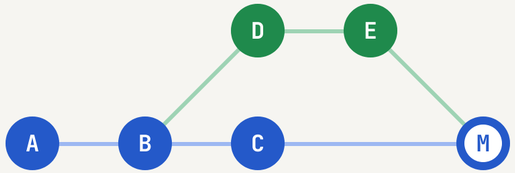
fast-forward



`git merge 브랜치`

main에 새 커밋이 없을 때 git merge가 알아서 해요

merge




```
git merge 브랜치
git pull origin main
```

pull할 때, push한 브랜치에 main을 받을 때

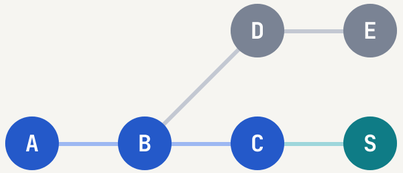
rebase



`git rebase main`

push 전, 내 브랜치를 최신 main 뒤로

squash




```
git merge --squash 브랜치
Squash and merge
```

작은 커밋 여러 개를 하나로, 뒤에는 -D

헷갈리면 merge를 써요. 다음 챕터 09 uv로 파이썬 시작하기에서는 파이썬 프로젝트를 만들어요.